

ERFS: Efficient Feature Graph Representation for Intrusion Detection Based on Flow Semantic Association

Yiqing Luo[✉], Mingshu He[✉], *Member, IEEE*, and Xiaojuan Wang[✉]

Abstract—The detection of network threats remains a fundamental task in cyberspace defense. Graphs are capable of capturing rich structural information and retaining content information. They are widely applicable in the field of threat detection. However, existing graph-based threat detection methods are computationally complex and challenging to apply in real-world network scenarios, where topologies may be incomplete or invisible. To address this, we propose ERFS, an efficient feature graph representation method for intrusion detection based on flow semantic correlation. This method aims to enhance the feature representation of network behavior. The proposed method first constructs a semantic correlation graph to depict the inherent semantic relationships within and between network flows. This is followed by the use of a messaging network to update and learn the information of graph nodes. The updated nodes are then aggregated into flow-level semantic feature representations for node classification, thereby achieving threat detection. The key innovation of this approach lies in graph construction and learning. The semantic correlation graph is constructed based on contextual correlation features and does not rely on topological information. It addresses the existing dependency issue of traditional graph construction methods on network topology. On the other hand, it introduces a small-window learning mechanism and a global sharing mechanism for edge weights. These mechanisms enable graph learning within a small-window range while globally sharing representations of identical nodes and edge weight values. It also addresses challenges in current graph construction methods, such as high computational complexity and low efficiency. Furthermore, the evaluation results indicate that this method achieved classification accuracies of 99.99%, 98.81%, 100%, 97.87%, and 99.64% on five public datasets (CIC IDS2017, CIC DOS2017, CIC DDoS2019, USTC-TFC2016, and ISCX VPN-nonVPN), respectively. These results surpass those of existing advanced models. The code is available on: <https://github.com/sixteen23333/ERFS>.

Index Terms—Threat detection, high efficiency, semantic association graphs, global sharing mechanisms.

Received 2 October 2024; revised 18 December 2024 and 11 February 2025; accepted 13 February 2025. Date of publication 18 February 2025; date of current version 10 December 2025. This work was supported by the National Natural Science Foundation of China under Grant 62402053 and 62227805. The associate editor coordinating the review of this article and approving it for publication was C.-H. Lee. (*Corresponding author: Mingshu He.*)

Yiqing Luo is with the School of Electronic Engineering, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: luoyiqing@bupt.edu.cn).

Mingshu He and Xiaojuan Wang are with the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: hemingshu@bupt.edu.cn; wj2718@bupt.edu.cn).

Digital Object Identifier 10.1109/TCCN.2025.3543357

I. INTRODUCTION

CYBERSPACE has become the second realm of human existence. Its security concerns have emerged as a new focal point of national interests. With the rapid advancement of the Internet and the widespread adoption of information technology, businesses, organizations, and individuals are increasingly confronted with cybersecurity threats. These threats include a wide range of malicious activities, such as hacker intrusions, phishing attempts, malware infections, data breaches, among others. These activities result in significant financial and reputational damage for the victims. As a result, detecting and uncovering threat behaviors is crucial to safeguarding cyberspace. Within the vast data flow carried by network activities, a notable characteristic is its immense scale, extensive temporal and spatial spans, and widespread distribution. At the same time, malicious behaviors use encryption, fragmentation, obfuscation, and disguise to deeply hide within this massive data deluge. This makes it exceedingly difficult to directly identify threatening behaviors through raw network flow data analysis.

To effectively differentiate between normal and abnormal network traffic, feature extraction and application have become essential. Features serve as crucial indicators for determining the normalcy of network traffic. They can also accurately identify abnormal patterns, malicious behaviors, and potential threats, thereby providing a solid foundation for threat detection. Feature extraction converts complex raw data into a more representative and easily processed form. This significantly enhances the efficiency of threat behavior recognition and detection. Among these, graph-based network behavior feature representation methods are widely used in the field of threat detection. By representing the internal semantic relationships of network flows as a graph structure, behavior can be analyzed accordingly. Compared to raw data, graph structures provide a more robust and abstract system view, making it more difficult for attackers to evade detection. Moreover, graphs provide richer semantics than raw data by preserving valuable relational information.

The current methods for representing network behavior using graph structures still face certain limitations in terms of both graph construction and scope. In the compositional approach, existing graph-based flow detection methods focus on learning node representations and aggregating messages in networks with known topologies [1], [2], [3]. However, in practical applications, network topologies are often not

readily available [4]. For instance, social networks may have incomplete or unmeasurable structures. Regarding the composition range, current methods typically involve constructing a graph for the entire database [5], [6], [7], [8], [9]. This approach facilitates better extraction of global features at the spatial level. Nevertheless, it also presents challenges related to computational complexity and time consumption, particularly due to the large number of edges.

In summary, we face two key challenges: (1) How to construct graph structures that represent behavioral association relationships in the presence of incomplete network topologies. (2) How to improve learning and testing efficiency of approaches that construct graphs based on entire databases.

To effectively tackle the task of threat detection and address the aforementioned challenges, this paper focuses on improving the composition method and graph learning approach. Regarding composition, a consistent correlation between network flow data and network behavior has been observed across various scenarios. The content correlation within flow data reflects underlying behavioral patterns, regardless of topological transmission information. Therefore, we aim to reduce reliance on the topological information of graph structure and instead explore network flow behavior characteristics through dimensions such as contextual relevance and semantic features. In terms of graph learning, current methods often require recalculating node information during testing. This adds complexity, consumes time, and leads to poor generalization. To address this, our efforts focus on facilitating information sharing between the training and testing processes. This allows the knowledge acquired from training nodes to be generalized to testing nodes without the need for recalculation during the testing phase. As a result, efficiency is improved and application costs are reduced.

Building on the ideas outlined above, this paper proposes an intrusion detection method based on flow semantic correlation. Unlike existing approaches that focus solely on connection information, this method starts with the original flow data and treats it as a specialized language. By analyzing the structural similarity between flow data and text data, we construct a semantic association graph to reveal underlying behavioral patterns in the network flow. Based on the constructed semantic association graph, this study aims to address the issues of complex calculations and low efficiency in existing graph learning methods. To achieve this, we design a global sharing mechanism for edge weights and a small-window learning mechanism. The global sharing mechanism enables the sharing of node representations and edge weights, eliminating the need for recalculation during testing. Furthermore, by treating each flow as a window unit, graph learning and updating unfold on a smaller scale based on messaging networks.

The main contributions of this paper are as follows.

- This paper proposes an efficient feature graph representation method for intrusion detection based on the semantic correlation of network flows. Drawing on the similarity between network flow data and natural language, the concept of semantic analysis is applied to network threat detection. Using semantic features as the foundation, the method learns and analyzes the semantic correlation

between flows. The proposed correlation features reveal the behavioral semantic characteristics of each type of network flow, which can serve as “fingerprints” to differentiate them. Unlike existing graph construction methods that focus on connection information during transmission, this method emphasizes the semantic feature representation of flow data. As a result, it reduces dependency on network topology and is suitable for real-world scenarios where topology is either invisible or incomplete.

- To address the issues of low efficiency in existing graph construction methods, we propose a global sharing mechanism for edge weights and a learning mechanism for a small window. This approach involves globally sharing the representations and edge weights of identical nodes and generating a graph structure for each network flow. Additionally, it connects byte nodes within a smaller window in the flow data, rather than directly and fully connecting all byte nodes in the database. Evaluation results demonstrate that this method achieves high accuracy rates with minimal iterative training requirements. It also ensures fast iteration training processes.
- This method has been evaluated on five public datasets, which include data from the Internet, the Internet of Things (IoT), and the encrypted Internet. The results demonstrate its ability to effectively detect threats in various scenarios. It exhibits exceptional performance in the IoT detection scenario, achieving a remarkable detection accuracy of 100%. Furthermore, the extracted semantic correlation features exhibit a distinctive recognition capability for various network threat behaviors. Each of these behaviors has unique behavioral and spatiotemporal characteristics. These threat behaviors include brute force, HULK, DoS, DDoS, and other types of attacks.

The remainder of the paper is organized as follows. Section II outlines the motivation, contributions, and model design considerations for adopting semantic analysis ideas in threat detection. Section III describes the overall architecture and design of the semantic association graph-based behavior analysis model. Section IV evaluates and validates the proposed model and discusses and analyzes the experimental structure. Section V presents the conclusions and discusses future work.

II. BACKGROUND AND RELATED WORK

A. Background

In existing cyberspace, it can be divided into public networks and dedicated networks for different scenarios based on the visibility of network flows, as shown in Fig. 1. The topology structure of the nodes in the private networks is invisible, and the private networks are connected to each other through the topology nodes of the public networks. In such a scenario, the detection and discovery of network threats can only be achieved by collecting and detecting flows on the public network topology path, and then tracing the source to find the initial threat effector node to contain and block the spread and development of threats.

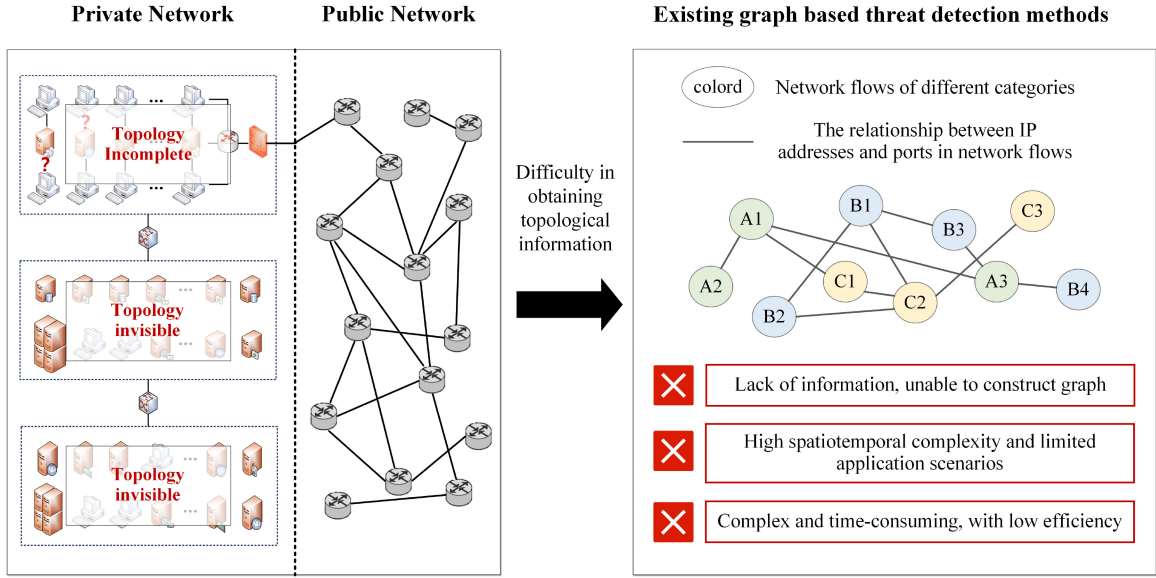


Fig. 1. The application scenarios and motivational ideas of this method.

Due to the invisibility of dedicated networks and the potential topological incompleteness, obtaining transmission information such as IP addresses and ports poses a significant challenge. However, the majority of existing graph-based threat detection methods rely on network topology structures that encompass IP addresses and port details. While IP address and port information offer robust behavioral indicators for distinguishing network traffic from diverse sources and destinations, they may not effectively fulfill threat detection tasks in scenarios where acquiring topological information proves challenging. For example, Zhang et al. [9] constructed a traffic trajectory map with traffic as nodes, with the edges being a combination of IP addresses and ports; Ge et al. [10] constructed a traffic behavior graph and defined nodes based on tuples of target IP and ports; Deng et al. [11] transformed the topological structure of flows into graphical representations for graph learning.

Therefore, graph based threat detection methods need to break away from the dependence on topological information and mine network behavior as features from another dimension. Simultaneously, endeavors are imperative to enhance graph computation and learning methodologies in order to bolster efficiency and mitigate the cost associated with method application.

From the initial network flow data, we have observed a consistent correlation between network flow data and network behavior across various scenarios. The intrinsic correlation within the flow data itself and among different flows reflects underlying behavioral patterns, independent of topological transmission information. Building upon this observation, our study aims to investigate the correlational features present within and between the original flow data. Time and spatial characteristics have been widely proven to have good representativeness for behavior. In terms of time series, we attempt to explore the correlation between data within each flow to describe the semantic composition structure of each

flow. As for space, considering the effectiveness and efficiency of graph learning, we seek to use edge weight sharing to assign global information to the data node representation in each flow, thereby obviating the necessity for recalculating node representations during testing while maximizing effectiveness and efficiency. By learning the semantic association graph constructed in this way, we can obtain the representation of each byte, and then restore the semantic representation of each network flow to determine behavior.

B. Related Work

1) *Research on Network Flow Detection Based on Context Features:* Network threats are often not isolated, but rather associated with a series of related events or behaviors. By extracting contextual features from network flows, the correlation and dependency between these events or behaviors can be revealed, thus providing a more comprehensive representation of network behavior. In existing research on context-based network flow detection, based on the method of feature extraction, it can be divided into two categories: using specific methods to extract features and using specific model algorithms to extract features.

Some studies have utilized text analysis techniques, such as N-Gram, TF-IDF, and PMI, for network flow detection to extract contextual features. Wang et al. [12], [13] employed the N-Gram word segmentation method for extracting semantic features in classification, demonstrating effective identification of malicious flows with minimal false alarms. AlEroud and Karabatis [14] incorporated time and location information in flow analysis to establish semantic links between suspicious activities and generate a semantic link network for preliminary prediction of suspicious nodes. However, the aforementioned research still exhibits limitations in terms of granularity and breadth of contextual feature learning; for instance, AlEroud et al.'s study is constrained by

a narrow scope of feature extraction focused on network flows transmitted between the same nodes, resulting in coarse feature granularity that fails to capture complete flow information.

In addition, several studies employ specific machine learning and deep learning models or algorithms to analyze time series data and uncover contextual features, such as RNN and LSTM. Dai et al. [15] proposed a novel cross-modal fusion model CMFTC that achieves superior performance through the integration of packet-level features, context, and flow-level patterns, with the extraction of contextual features based on packet-level features using stacked extractors consisting of multiple blocks. Gao et al. [16] addressed the anomaly detection issue in IoT scenarios by introducing a memory mechanism to suppress the model's generalization ability and designing a memory-enhanced time series autoencoder (TSMAE), which incorporates long short-term memory (LSTM) for decoding and encoding time series data to obtain contextual features. Li et al. [17] obtained initial feature representations of payloads based on word embedding methods and subsequently proposed a network flow context learning algorithm for payload detection. Ling et al. [18] and Wu et al. [19] utilized image transformation of network flow data to conduct a comprehensive analysis of semantic relationships within the graph. Their experiments demonstrated that models incorporating graph semantic relationships outperformed those focusing solely on graph structural information. Ling et al. [18] employed hierarchical learning to represent executable files by capturing the structural semantics of each function, while Wu et al. [19] applied BERT and Graph Isomorphism Network (GIN-JK) to capture both semantic and structural characteristics of control flow graphs. Although these representations effectively capture local behavioral patterns over time, they may overlook spatial global information in network behavior analysis.

In conclusion, the context features offer a more comprehensive representation of online behaviors. Nevertheless, current feature extraction methods still possess certain limitations in terms of the scope and dimensionality of features. Some studies solely concentrate on the contextual features of individual online flows, with feature dimensions restricted to time while disregarding spatial global information.

2) *Research on Graph-Based Network Flow Detection:* Graphs often provide a good description of the global relationships in hierarchical structures. In graph-based methods, most of them construct heterogeneous relationship graphs based on the topological structure between network flow, and use graph learning models to train and classify heterogeneous graphs, as shown in Table I. Considering the focal points of existing research, we will analyze from two perspectives: graph embedding features and graph learning methods.

In terms of graph embedding features, effectively inputting high-dimensional graph structures into neural networks and other learning models is challenging. Therefore, existing research designs graph embedding to discover low-dimensional vector representations of high-dimensional graphs, achieving dimensionality reduction and representation of the graph structure by extracting features such as the overall structure of the graph and relationships between nodes.

Some studies are dedicated to examining the overall characteristics and structural properties of graphs. Cao et al. [20] and Duan et al. [21] simultaneously consider the embedding features of graph structure in both time and space dimensions. In their study, Cao et al. [20] utilized data plane programmable SDN and in-band network telemetry to sample and perceive the state of switches. They inputted this network state into a spatiotemporal graph convolutional detection model to accurately identify the switches through which DDoS attacks pass. Duan et al. [21] proposed a semi-supervised intrusion detection method utilizing the dynamic line graph neural network to transform the network flow into a sequence of spatio-temporal graphs. This approach employs Dynamic GNN to extract spatial information from individual discrete snapshots and capture the contextual evolution of communication between IP pairs across consecutive snapshots. Hu et al. [22] proposed a novel early network intrusion detection method, Graph2vec+RF, which is based on graph embedding technology. This approach involves constructing a flow graph for each bidirectional network flow from initial interactive packets and utilizing graph embedding technology to learn vector representations of the flow graphs, followed by classification using random forest. Wang et al. [23] proposed a hybrid analysis model BotMark for detecting zombie networks, which extracted 15 statistical flow-based traffic features and 3 graph-based features. The behaviour was analyzed through a combination of detection results based on similarity score, stability score, and anomaly score. In addition, some studies focus on the relationship between nodes to form graph embeddings. Zhao et al. [6] specifically focused on the detection of robots and constructed a multi-attribute heterogeneous graph by modeling IP, response, and other network flow objects. They employed a node embedding method based on weight learning to understand the interaction patterns among robots and incorporated them into a weighted similarity graph. Finally, the heterogeneous graph was input into a graph convolutional model for learning.

As for graph learning methods, the traditional GCN model is still one of the mainstream learning models. Current research focuses on improving classic models and combining multiple models to embed node and topological structure information more effectively. Zheng et al. [5] developed a modified GCN model for detecting encrypted malicious traffic, utilizing IP addresses as nodes and flows as edges to construct a traffic trajectory graph. They also employed an enhanced GCN to effectively capture and learn the structural and attribute information within the graph. Chen et al. [24], Chen et al. [25], and Shen et al. [26] combined multiple neural network models to jointly extract graph structure features. Chen et al. [24] utilized a transformer-based architecture for automatic learning of graph structures, graph convolutions, and modeling time-dependent relationships. They introduced influence-propagation convolution in graph learning and proposed multi-branch attention techniques to enhance inference speed. Chen et al. [25] introduced a novel flow-graph expression model WFF, which involves the transformation of bidirectional flows into graphs based on Markov properties within flows. The proposed model

TABLE I
ANALYSIS OF EXISTING GRAPH BASED THREAT DETECTION METHODS

Literature	Composition range	Graph embedding features	Graph learning methods	Advantage	Disadvantage
ST-GCN [20]	Entire database	Time and spatial characteristics	GCN, CNN	Multiple feature dimensions	The feature range is large and lacks details
DLGNN [21]		Time and spatial characteristics	DLGNN	Multiple feature dimensions require fewer labels	High complexity
Bot-AHGCN [6]		Similarity features	GCN	Fine feature granularity	Low efficiency
Graph2vec+RF [22]		Graph2vec features	skip-gram	Less occupancy and consumption	Figure represents a single
BotMark [23]		number of nodes and edges	egonet	automatic detection	High complexity
GCN-ETA [5]		adjacency matrix	GCN	Simple model	Figure represents a single
GTA [24]			GCN, transformer	Fast reasoning speed	Figure represents a single
WFF-EGNN [25]			GCN/GGNN/CapsGNN	More complete expression of relationships	High complexity
GraphDApp [26]			GNN, MLP	Identify encrypted flow	Unable to update features

implements WFF-based classification using graph convolution networks, gated graph neural networks, and capsule graph neural networks, respectively. Shen et al. [26] developed traffic interaction graphs using bidirectional client-server interaction features and employed multilayer perceptrons to construct GNN-based classifiers.

In summary, the majority of current graph-based flow detection methods commence from the perspective of flows and packets, analyzing the spatiotemporal relationships of observed flows in the network over a period of time and representing flows as graph structures. With regards to graph embedding, existing methods predominantly rely on spatiotemporal features for identifying flows, which is somewhat reliable but entails computing statistical features for each individual flow, resulting in a limited effective range for flow detection. In terms of graph learning, current methods often utilize unimproved classic graph learning models such as GCN, which tends to exhibit high complexity issues.

III. METHODOLOGY

The model mainly comprises a semantic association graph and messaging network, as shown in Fig. 2. This method performs a unified and standardized semantic field-slicing process on the network flow based on the characteristics of its original semantic structure. Then, for each network flow, a graph structure containing semantic association relations is constructed with the smallest unit, the byte, as the granularity and the packet as the feature extraction unit. The node representation and edge weights are shared through a global sharing mechanism. Subsequently, the message passing mechanism (MPM) is used to update and learn the graph node feature information, and the self-attention tool is used to aggregate the updated node features into flow-level semantic feature representations, thereby completing behavior detection and recognition. In Fig. 2, nodes of different colors represent additional semantic data of network traffic, the solid black lines between nodes are byte data edges, and $R(x)$ represents

the representation of x after learning updates by the messaging network. During the training and testing process, separate semantic correlation graphs will be constructed for the training data and testing data. In the training phase, this approach involves inputting the constructed semantic correlation graph into an information propagation network for learning, and saving the updated byte node representations and byte-by-byte edge weight information in shared representation matrices and shared weight matrices. During the testing phase, the semantic correlation graph constructed for the test data will acquire corresponding information from the shared representation matrices and shared weight matrices as initial node representations and edge weights. These will then be input into an information propagation network for learning, aggregating the updated byte node representations into a flow-level feature representation for classification. The overall system diagram and flowchart are shown in Fig. 3.

A. Unified Data Representation

Based on semantic analysis, certain pre-processing operations must first be performed on the network flow data to realize treating the flow data as text data and then classifying them based on their semantic association features.

According to the heavy-tailed feature of Internet traffic, we use the data from the previous part of each flow to analyze the data effectively. Therefore, in this method, we plan to use the first 500 bytes of data in each flow to make the technique more efficient while ensuring the validity of the data. In similarity to the procedures followed by Wang et al. [13] and Li et al. [17], we will adopt the strategy of directly utilizing the original flow hexadecimal number sequence data to maintain the data's semantic and content integrity. For graph structures, when the length of a flow data exceeds or equals 500 bytes, we will truncate it and directly capture the first 500 bytes of data. Data that is more minor than 500 bytes will not be processed at this time. Additionally, to further simulate scenarios with incomplete network topology in real networks, we removed

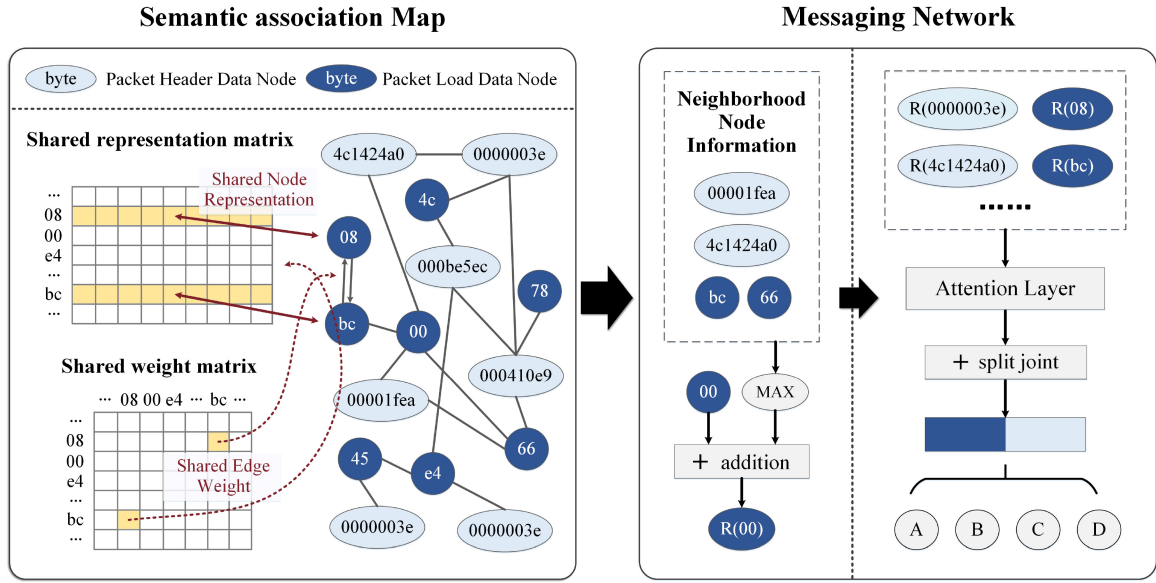


Fig. 2. Model architecture.

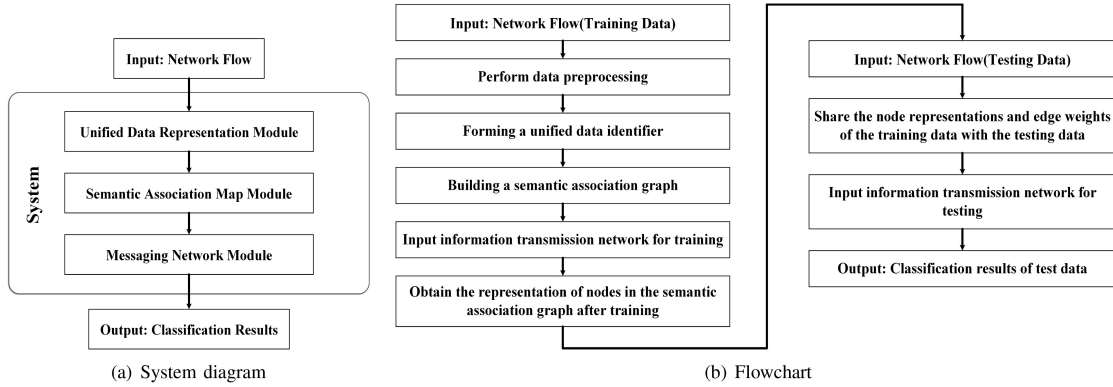


Fig. 3. Overall system diagram and flowchart.

the MAC address information in the data link layer and the IP address information in the IP layer to achieve anonymity.

The traditional view of network traffic examines packets as a collection of higher-level headers such as IP, TCP, and UDP. Each header has semantic fields, such as IP time to live, TCP port number, and UDP length fields. The standard semantic representation of network traffic collects these semantic fields into a single representation. We propose a data slicing method to make semantic analysis more applicable in network threat detection. This method preserves the original semantics of the network flows, maximizing the possibility of maintaining the original data structure while standardizing the processing of the flow data structure, as shown in Fig. 4. To better characterize the semantic structure of the network flow, we retain the original semantic structure of the packet headers and perform the word separation process accordingly, and for the payload content, we slice and dice using a fixed window size W_d .

B. Semantic Association Map

Combining the limitations of existing graph-based threat detection methods in feature construction, we first consider

avoiding connection features in network flow transmission and instead start from raw flow data to represent the persistent data correlation between network behaviors using a graph structure. Based on the similarity between text data structure and flow data structure, we use text analysis ideas to construct the graph.

Based on the inherent correlation between network flow and behaviour, it is observed that flows within the same behaviour category exhibit certain similarities at the data level. Considering the structural characteristics of flow data, we propose to employ semantic analysis methods to investigate the correlation between data. Text analysis is a method used in the fields of natural language processing and computational linguistics, with the goal of achieving a comprehensive understanding and interpretation of textual data through techniques such as statistics, pattern recognition, and machine learning. Previous studies have utilized text analysis methods to explore features related to behavioural relationships [12], [13], [27], [28]. Recent studies have uncovered a significant resemblance between flow protocol data, mobile flow data, and natural language [13], [27]. The n-gram word sequences in flow exhibit highly skewed frequency rank distributions similar to those found in natural language [13]. As a result, it is viable

	Packet Header1				Packet Data1	Packet Header2	Packet Data2	
	Timestamp (4B)	Timestamp (4B)	Caplen (4B)	Len (4B)	Data			
Flow1	4c1424a0	00001fea	0000003e	0000003e	08 00 45 00 00 ...	c4cb6500 ...	78 e4 00 6c ...	
Flow2	4c17d46d	000410e9	0000003e	0000003e	08 00 45 00 00 ...	4fc4c72b ...	25 f1 72 a5 ...	
Flow3	4c17d5f1	000be5ec	0000003e	0000003e	08 00 45 00 00 ...	4fc48892 ...	78 e4 00 6b ...	
.....								

Fig. 4. Unified expression form for network flow data (payload slice with fixed window size $N=2$ for example).

to consider flow data as text data for the purpose of text analysis, enabling extraction of fine-grained semantic information contained within the flow for behaviour identification. For instance, features such as transmission time and packet length can be derived from packet header data, while behavioural content features can be extracted from the overall structure of payload data.

For each network flow, based on its smallest unit, bytes, we model the entire corpus of flow data and construct a semantic association graph with byte data as nodes and their contextual association relations as edges, using the flow as a unit.

Each byte in a packet is treated as a node, and edges are constructed between adjacent nodes. Furthermore, to address the poor scalability and high complexity of traditional graph learning models, we globally share exact node representations and edge weights. Boundaries with fewer than k occurrences in the training set are evenly mapped to a “common” edge, enabling full parameter training. Finally, the constructed graph is represented as $G = (V, E)$, where V is the set of nodes and E is the set of edges, as follows:

$$V = \{v_i | i \in [1, l]\}. \quad (1)$$

$$E = \{e_{ij} | i \in [1, l], j \in [i - p, i + p]\}. \quad (2)$$

where v_i is the i -th byte data node, l is the number of byte data in that network flow, e_{ij} is the edge between nodes v_i and v_j , and p denotes the number of adjacent words connected to each word in the graph.

C. Messaging Network

The message passing paradigm is a paradigm that aggregates neighboring node information to update central node information, extending the convolutional operator to the domain of irregular data and connecting graphs with neural networks. Due to its powerful information propagation and integration capabilities, as well as its flexible and efficient characteristics, the message passing paradigm has been widely applied in fields such as social networks, bioinformatics, and finance. Based on this, this method adopts the message passing mechanism proposed by Gilmer et al. [29] to learn and update graph structures.

MPM initially gathers information from neighboring nodes and adjusts its representation based on both its original data

and the gathered information. The process of message-passing can be formulated as follows:

$$M_n = \max_{a \in N_n^p} e_{an} v_a. \quad (3)$$

$$v'_n = (1 - \omega_n) M_n + \omega_n v_n. \quad (4)$$

M_n is the message received by node n from its neighbors; $\max$ is a reduction function that combines the maximum values in each dimension to form a new vector as the output. N_n^p denotes the node with the p words closest to n in the original text. e_{an} is the edge weight from node a to node n , which can be updated during training; v_n denotes the previous representation of node n . ω_n is the node n which is a trainable variable that indicates how much v_n information should be retained. v'_n denotes the updated representation of node n .

The MPM mechanism causes a node's representation to be influenced by its neighbors, meaning that the representation can bring information from the surrounding context. Therefore, the semantic meaning of the same data byte can be determined by the weighted information from the neighbors. Additionally, the parameters of the text-level graphs are extracted from a globally shared matrix, meaning that these representations can also encode global information like other graph-based models.

After being updated sufficiently, the graph nodes are aggregated into flow-level semantic representations using a self-attentive mechanism. The concept behind this is that each data byte plays a role in characterizing part of the semantics in the network flow, while the critical node data should have a more explicit contribution. Ultimately, the representation of all nodes in a flow is used to predict behavioral labels:

$$y_i = \text{softmax} \left(\text{Relu} \left(W \sum_{n \in N_i} v'_n + b \right) \right). \quad (5)$$

where W is the matrix that maps the vectors to the output space, N_i is the set of nodes of the network flow, and b is the deviation.

Algorithm 1 describes the training and testing stages of this method.

IV. EVALUATION

The proposed method was evaluated using five publicly available datasets: CIC IDS2017, CIC DOS2017, CIC DDoS2019, USTC TFC2016, and ISCX VPN-nonVPN. One

Algorithm 1 Training and Testing Phase

Input: Training Graph $G_{train}(V_{train}, E_{train})$, Nodes $a, n \in V_{train}$, Training labels $\{Y_i, i \text{ is the sample of the training set}\}$; Testing graph $G_{test}(V_{test}, E_{test})$, Nodes $b, m \in V_{test}$, Testing labels $\{Y_j, j \text{ is the sample of the testing set}\}$; L is the loss function

Output: Prediction Results $\{y_i, i \text{ is the sample of the training set}\}$, $\{y_j, j \text{ is the sample of the testing set}\}$

```

1: for  $epoch = 1, 2, \dots$  do
2:   for  $n \in N_i$  do
3:      $M_n \leftarrow \max_{a \in N_n^p} e_{an} v_a$ 
4:      $v'_n \leftarrow (1 - \omega_n) M_n + \omega_n v_n$ 
5:   end for
6:    $y_i = \text{softmax}(\text{Relu}(W \sum_{n \in N_i} v'_n + b))$ 
7:    $L = -Y_i \times \log(y_i) - (1 - Y_i) \times \log(1 - y_i)$ 
8:   Update  $\{e_{an}, a, n \in V_{train}\}$ ,  $\{v'_n, n \in V_{train}\}$ ,  $\{\omega_n, n \in V_{train}\}$ 
9: end for
10: for Batch  $b$  in  $V_{test}$  do
11:   for  $m \in N_j$  do
12:      $e_{bm} \leftarrow e_{an}$ 
13:      $V_m \leftarrow V'_n$ 
14:      $\omega_m \leftarrow \omega_n$ 
15:      $M_m \leftarrow \max_{a \in N_m^p} e_{bm} v_b$ 
16:      $v'_m \leftarrow (1 - \omega_m) M_m + \omega_m v_m$ 
17:   end for
18:    $y_j = \text{softmax}(\text{Relu}(W \sum_{m \in N_j} v'_m + b))$ 
19: end for

```

dataset consists benign traffic and multiple types of malicious traffic in an Internet scenario, two datasets contain various types of DoS and DDoS attacks in an IoT scenario, and the remaining dataset includes encrypted and unencrypted raw traffic in the encrypted Internet scenario.

A. Evaluation Setup

1) *Dataset*: We validated the method using the CIC IDS2017, CIC DOS2017, CIC DDoS2019, USTC TFC2016, and ISCX VPN-nonVPN datasets. The detailed information of the dataset is shown in Table II.

CIC IDS2017 dataset contains both benign and up-to-date common attack data, with mild background traffic generated by the Canadian Institute for Cyber Security using the B-Profile system to characterize abstract behaviors of human interactions, constructed for 25 users based on HTTP, HTTPS, FTP, SSH, and email protocols. The attack behavior data include Brute Force FTP, Brute Force SSH, DoS, Heartbleed, Web Attack, Infiltration, Botnet, DDoS, etc.

CIC DOS2017 dataset used different tools to make four attacks on the ten web servers with the highest connections in the ISCX dataset. It obtained eight separate application layer DoS attack tracks, with a total size of 4.6 GB. DoS attacks are one of the common attacks in IoT networks, and the CIC DOS2017 dataset validates the possible application of this model in IoT scenarios.

CIC DDoS2019 dataset contains benign traffic and the most up-to-date common DDoS attacks (volumetric and application: low volume, slow rate), resembling real-world data. This dataset includes modern reflective DDoS attacks such as PortMap, NetBIOS, LDAP, MSSQL, UDP, UDP-Lag, SYN, NTP, DNS, and SNMP. DDoS attacks are also one of the common attacks in IoT networks, and the CIC DDoS2019 dataset validates the possible application of this model in IoT scenarios.

USTC TFC2016 dataset contains ten types of malicious traffic under eight kinds of applications, including Zeus, Virus, Tinba, etc. This dataset consists of two parts, including encrypted and unencrypted raw traffic. The first part contains 2015 types of malicious software traffic from public websites collected by researchers at the Czech Technical University (CTU) from a natural network environment from 10 years to 2011. The second part contains ten types of regular traffic collected using the network traffic simulation device IXIA BPS. The addition of the USTC TFC2016 dataset can help verify the potential application of this model in encrypted Internet scenarios.

ISCX VPN-nonVPN dataset is widely used in encrypted network flow classification. This data set captures a regular and a VPN-based session for each traffic type. Therefore, it includes seven kinds of normal encrypted traffic, such as VoIP, VPN-VoIP, P2P, VPN-P2P, and seven types of protocol-encapsulated traffic, totaling 28GB. The ISCX VPN-nonVPN dataset validates the possible application of this model in encryption scenarios.

2) *Parameter Setting*: We provided training and testing sets for all datasets, with the training set randomly split into a 7:3 ratio for training and validation. The performance on the validation set was adjusted based on hyperparameters, including setting the initial learning rate of Adam optimizer to 0.0005, L2 weight decay to 0.0001, neighborhood connectivity parameter p to 3, and weight threshold parameter k to 2.

3) *Evaluating Metrics*: In this work, we used accuracy, precision, recall, and F1-score to evaluate the classification ability of the proposed method. These indicators are defined as follows:

Accuracy (ACC) is the ratio of the correctly labeled subjects to the whole pool of subjects, i.e.,

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}. \quad (6)$$

Precision (PRE) is the ratio of the correctly positive labeled subject to all positive labeled subjects, i.e.,

$$\text{Precision} = \frac{TP}{TP + FP}. \quad (7)$$

Recall (REC) is the ratio of the correctly positive labeled to all who are positive in reality, i.e.,

$$\text{Recall} = \frac{TP}{TP + FN}. \quad (8)$$

F1 score (F1) is the harmonic mean (average) of precision and recall, and we use a microscopic approach to this calculation, i.e., Macro-F1.

$$F1 - \text{Score} = \frac{2 * \text{Recall}_m * \text{Precision}_m}{\text{Recall}_m + \text{Precision}_m} \quad (9)$$

TABLE II
DATASET DETAILS

Dataset	Normal/ attack	Number of samples	Normal /attack	Number of samples
CIC IDS2017	Benign	496922	Hulk	48734
	Botnet	83865	Infiltrating	1408
	BruteForce	1507	XSS	652
	DDoS	119293	SlowHttp	37958
	GoldenEye	30379	SlowLoris	37357
	FTP	98192	SSH	80441
	HeartBleed	42026	SqlInjection	4129
CIC DOS2017	PortScan	410470		
	slowread	20452	slowbody	36762
	slowloris	21099	Hulk	60601
	slowheaders	45848	goldeneye	89814
CIC DDoS2019	rudu	21261	ddossim	8480
	LDAP	30000	SYN	2719
	MSSQL	2268	UDP	30000
	NetBIOS	30000	UDP-Lag	30000
USTC TFC2016	PortMap	2311		
	Zeus	12032	Miuref	10051
	Virus	13158	Htbot	12657
	Tinba	16213	Geodo	13804
	Shifu	15772	Cridex	16335
ISCX VPN-nonVPN	Nsis-ay	1115	Benign	17178
	Neris	17008		
	Chat	12000	P2P	2758
	Email	6598	Streaming	5164
	FileTransfer	7949	VoIP	12000

$$Precision_m = \frac{\sum_{i=1}^n Precision_i}{n} \quad (10)$$

$$Recall_m = \frac{\sum_{i=1}^n Recall_i}{n} \quad (11)$$

B. Evaluation Results

We used the Python language for the evaluations Pytorch for the network framework, and all evaluations were run on an NVIDIA GeForce RTX 2080 Ti graphics card. The experiment is divided into five parts: data representation analysis, parameter optimization analysis, overall comparison analysis, learning efficiency analysis, and application scenario analysis. The first two parts explore the possibility of model optimization in terms of data and parameters, while the latter three mainly focus on model performance and analyze from the aspects of overall comparison, model efficiency, and model application scenarios.

1) *Data Expression Analysis*: The representation of data plays a crucial role in behavior characterization. The effectiveness of language association graphs in representing behavior is directly influenced by different data ranges and segmentation methods. Therefore, this section begins with the standardized expression of data and examines the efficacy of data representation range and segmentation method, encompassing both the data packet header and payload. The objective is to identify the optimal approach for data representation and investigate the impact of various data representation methods on behavior characterization.

TABLE III
ANALYSIS OF THE RANGE OF DATA EXPRESSION

Dataset	Data Range	Acc	Pre	Rec	F1
CIC IDS2017	Header	0.1652	0.0690	0.1614	0.0907
	Payload	0.5573	0.5650	0.5584	0.5604
	Header+Payload	0.9999	0.9999	0.9999	0.9999
CIC DOS2017	Header	0.1410	0.0695	0.1409	0.0833
	Payload	0.4994	0.5046	0.4994	0.4997
	Header+Payload	0.9881	0.9882	0.9880	0.9880
CIC DDoS2019	Header	0.4471	0.2367	0.4255	0.2922
	Payload	0.6301	0.5209	0.6372	0.5594
	Header+Payload	1.0000	1.0000	1.0000	1.0000
USTC TFC2016	Header	0.1370	0.0816	0.1369	0.0989
	Payload	0.9777	0.9778	0.9776	0.9777
	Header+Payload	0.9787	0.9788	0.9786	0.9787
ISCX VPN-nonVPN	Header	0.2631	0.2123	0.2345	0.2178
	Payload	0.6768	0.7080	0.6951	0.7006
	Header+Payload	0.9964	0.9961	0.9967	0.9964

TABLE IV
ANALYSIS OF PACKET HEADER DATA EXPRESSION SLICING METHODS

Dataset	Window	Acc	Pre	Rec	F1
CIC IDS2017	2	0.9998	0.9998	0.9998	0.9997
	4	0.5671	0.5721	0.5687	0.5699
	8	0.9999	0.9999	0.9999	0.9999
CIC DOS2017	16	0.6109	0.6178	0.6112	0.6140
	2	0.9919	0.9919	0.9919	0.9919
	4	0.5160	0.5180	0.5159	0.5154
CIC DDoS2019	8	0.9881	0.9882	0.9880	0.9880
	16	0.5525	0.5588	0.5525	0.5544
	2	1.0000	1.0000	1.0000	1.0000
USTC TFC2016	4	0.9421	0.9397	0.9355	0.9369
	8	1.0000	1.0000	1.0000	1.0000
	16	0.9373	0.9336	0.9307	0.9316
ISCX VPN-nonVPN	2	0.9727	0.9726	0.9726	0.9726
	4	0.9621	0.9636	0.9621	0.9623
	8	0.9787	0.9788	0.9786	0.9787
ISCX VPN-nonVPN	16	0.9674	0.9673	0.9673	0.9673
	2	0.9938	0.9940	0.9940	0.9939
	4	0.8260	0.8292	0.8331	0.8307
ISCX VPN-nonVPN	8	0.9964	0.9961	0.9967	0.9964
	16	0.8892	0.8894	0.8907	0.8897

a) *Analysis of the range of data expression*: The raw network flow data is organized into packets, each comprising a header and payload. Following the application of our proposed method for unified representation of network flow data processing, we specifically selected three types of data ranges: packet headers, payloads, and complete contents encompassing both headers and payloads. The model's detection performance was assessed on various datasets for each corresponding data range as presented in Table III. It is important to note that when evaluating the detection performance of a specific segment of the data representation range, other factors such as data segmentation methods and model parameters were kept consistent to control variables.

Overall, the payload data possesses a greater capacity for expressing behavioral information compared to the header

TABLE V
ANALYSIS OF PAYLOAD DATA EXPRESSION SLICING METHODS

Dataset	Window	Acc	Pre	Rec	F1
CIC IDS2017	2	0.9999	0.9999	0.9999	0.9999
	4	0.4328	0.4586	0.4287	0.4324
	8	0.1765	0.0849	0.1724	0.0931
	16	0.1879	0.0977	0.1836	0.1055
CIC DOS2017	2	0.9881	0.9882	0.9880	0.9880
	4	0.2512	0.3138	0.2512	0.2489
	8	0.1250	0.0156	0.1250	0.0277
	16	0.1250	0.0156	0.1250	0.0277
CIC DDoS2019	2	1.0000	1.0000	1.0000	1.0000
	4	0.8197	0.8304	0.8080	0.8123
	8	0.1501	0.0214	0.1428	0.0372
	16	0.1501	0.0214	0.1428	0.0372
USTC TFC2016	2	0.9787	0.9788	0.9786	0.9787
	4	0.8858	0.8947	0.8857	0.8827
	8	0.0909	0.0082	0.0909	0.0151
	16	0.0909	0.0082	0.0909	0.0151
ISCX VPN-nonVPN	2	0.9964	0.9961	0.9967	0.9964
	4	0.5226	0.5555	0.5367	0.4984
	8	0.3399	0.2148	0.3119	0.2207
	16	0.3509	0.2685	0.3089	0.2596

data. There exists a significant differentiation between standard and malicious behaviors in terms of packet transmission content, which can be leveraged to enhance the identification of network behaviors through the construction of a semantic correlation graph aimed at characterizing its data features.

b) *Analysis of data representation cuts*: The data packet header contains packet-level statistics, including timestamps and lengths. Following the unified representation proposed in this method, we selected segmentation windows of 2, 4, 8, and 16 to assess the model's detection performance on various datasets with different packet header segmentation methods, as presented in Table IV. It is important to note that when examining the detection performance of a specific data segmentation method, other factors such as data range and model parameters are kept constant to ensure controlled variables.

Table IV illustrates that the header has a specific data expression structure. Although fixed window sizes of 2 and 4 provide more fine-grained data analysis, they also destroy the original data expression structure, which can lead to poor detection results. In the unified expression of the header data, each field contains specific semantic information, most of which is statistical numerical information, such as storage length, timestamp high and low bits, etc. Slicing them according to a specific fixed-size window will not directly change the information difference between the data, and the network behavior can still be identified by the statistical characteristics between the data. Therefore, the impact of different slicing window sizes on the semantic slicing of data is limited. In the comprehensive representation of header data, each field contains specific semantic information, predominantly comprising statistical numerical data such as storage length and timestamp high and low bits. Slicing these fields according to a fixed-size

window does not significantly alter the informational variance between the data, and the behavior can still be identified by the statistical characteristics between the data. Thus, the impact of varying slicing window sizes on semantic data slicing is minimal.

The payload of a data packet refers to the specific content being transmitted across the network. To investigate the impact of different data segmentation methods on packet payload data, we utilized the unified representation proposed in this method to process network flow data. Subsequently, we selected four distinct segmentation windows (2, 4, 8, and 16) and assessed the model's detection performance on various datasets using different packet payload segmentation methods, as presented in Table V. It is important to note that when examining the detection performance of a particular data segmentation method, other factors such as data range and model parameters are kept consistent to control variables.

Overall, in comparison to header, the size of the payload slicing window can significantly impact the effectiveness of detecting network behavior. Although payload data lacks a specific structural expression, its detection accuracy is enhanced through a more fine-grained data slicing approach. In a unified representation of payload, the original transmission content is processed using a minimum grain size window 2, similar to previous research by [12], [27], which involves analyzing payload content and extracting statistical and content features for classifying and identifying network behavior. For instance, attacks such as DDoS and DoS exhibit significant feature information in both temporal and spatial dimensions of the payload; therefore, load feature information can be extracted with maximum granularity through semantic slicing of payload information.

2) *Parameter Optimization Analysis*: A crucial aspect of training graph neural networks involves fine-tuning the hyperparameters to balance variance, bias, and classification performance. Therefore, this section focuses on analyzing the number of neighboring words (p) connected to each word in the semantic association graph and the weight threshold (k) associated with shared edges. The model effect is analyzed explicitly by setting the magnitude of the values of the different parameters and adopting precision as a performance metric.

The network flow classification effects under different numbers of neighborhood connections parameter p and other weight threshold parameter k are shown in Fig. 5(a) and Fig. 5(b). The findings suggest that optimal byte co-occurrence information is achieved with a neighborhood connection window size of 3. A window size that is too small fails to capture sufficient global word co-occurrence information, while a window size that is too large may introduce connections between nodes with weak associations. Furthermore, the weighting threshold assigned to joint edges can be effectively balanced for training the learning center of gravity when set at 2, thereby enhancing learning efficiency without compromising feature learning effectiveness.

3) *Baseline Comparative Analysis*: This section presents a comparison of the proposed method with several classical machine learning models, including CNN, Lenet, Resnet, and

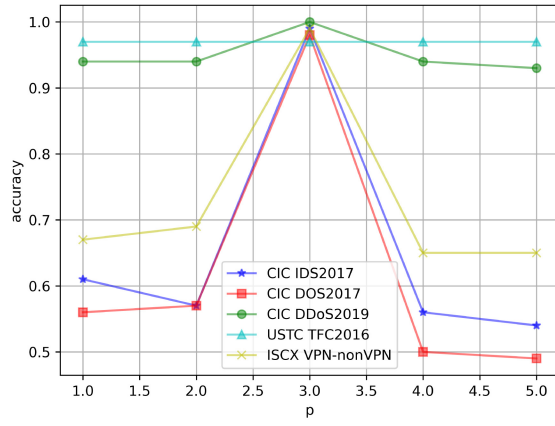
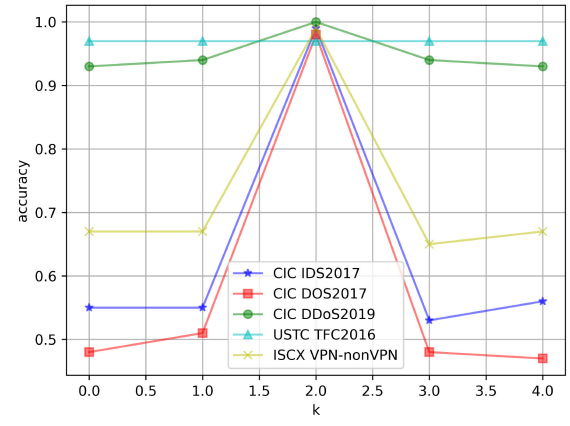
(a) neighborhood connections parameter p (b) weight threshold parameters k

Fig. 5. Analysis of model effects under different parameters.

TABLE VI
COMPARATIVE ANALYSIS RESULTS BETWEEN THIS MODEL AND
BASELINE MODELS

Dataset	Method	Acc	Pre	Rec	F1
CIC IDS2017	CNN	0.9575	0.9572	0.9572	0.9576
	Lenet	0.9114	0.9122	0.9110	0.9113
	Resnet	0.9080	0.9086	0.9076	0.9079
	RNN	0.9306	0.9316	0.9304	0.9305
	Our	0.9999	0.9999	0.9999	0.9999
CIC DOS2017	CNN	0.4791	0.4765	0.4790	0.4820
	Lenet	0.6162	0.6180	0.6180	0.6160
	Resnet	0.813	0.812	0.813	0.814
	RNN	0.8583	0.8591	0.8560	0.8582
	Our	0.9881	0.9882	0.9880	0.9880
CIC DDoS2019	CNN	0.9021	0.7942	0.8090	0.8016
	Lenet	0.8616	0.7918	0.7984	0.7950
	Resnet	0.9252	0.9020	0.9058	0.9040
	RNN	0.9126	0.8133	0.8087	0.8112
	Our	1.000	1.000	1.000	1.000
USTC TFC2016	CNN	0.9093	0.9080	0.9071	0.9075
	Lenet	0.8790	0.8700	0.8780	0.8790
	Resnet	0.8990	0.8980	0.8990	0.8984
	RNN	0.9164	0.9152	0.9140	0.9146
	Our	0.9787	0.9788	0.9786	0.9787
ISCX VPN-nonVPN	CNN	0.7940	0.7680	0.7890	0.7940
	Lenet	0.6470	0.6400	0.6320	0.6490
	Resnet	0.8170	0.8110	0.8070	0.8180
	RNN	0.7060	0.6880	0.6760	0.7120
	Our	0.9964	0.9961	0.9967	0.9964

RNN. Following parameter tuning, the learning rate for all baseline models is set to 0.01, with a batch size of 200. The network depths for the CNN, Lenet, Resnet, and RNN models are determined to be 7, 5, 18, and 2 respectively. It is important to note that the original network flow data utilized by both this method and all baseline models for comparison remains consistent. The experimental results are detailed in Table VI. Based on the experimental results, it is evident that this model demonstrates superior detection

performance compared to various other models across all three datasets. The detection accuracy reaches 99.99%, 98.81%, 100%, 97.87%, and 99.64% for the CIC IDS2017, CIC DOS2017, CIC DDoS2019, USTC-TFC2016, and ISCX VPN-nonVPN datasets, respectively.

Overall, the CNN and Lenet models utilize convolutional kernels for information extraction from images, while the Resnet model incorporates skip connections within the CNN architecture to facilitate learning of residual mappings. The RNN model is capable of exploring both temporal and semantic information in the data.

The baseline models, such as CNN, Lenet, and Resnet, aim to map the original network flow data to a new data space in order to analyze the differences in feature dimensions between normal and abnormal network flows. These models use convolutional layers to extract feature representations with spatial characteristics, while RNN models use recurrent layers to extract feature representations with temporal characteristics. Similarly, this method also maps the data to a new feature space but differs from the baseline methods by extracting semantic features containing temporal relationships and transmission content through constructing a semantic association graph. Subsequently, it updates graph information through message passing networks to extract feature representations with spatial characteristics. This detection mechanism allows for simultaneous extraction of time and space dimension data feature representations, providing a more comprehensive pattern representation.

4) *Comparative Analysis of Other Methods:* To further investigate the efficacy of this approach, in addition to the baseline model, this section compares it with several advanced network threat detection methods. The original network flow data used for comparison remains consistent, and parameters such as learning rate and batch size are set according to the optimal parameters proposed by the model. The experimental results are presented in Table VII.

From the presented results, it can be observed that the detection performance of this model across all five datasets exceeds that of all other evaluated models. As a whole, threat detection models can be divided into three parts: data, feature

TABLE VII
COMPARATIVE ANALYSIS RESULTS BETWEEN THIS MODEL AND STATE-OF-THE-ART MODELS

Dataset	Method	Data Characteristic	Learning Modes	Acc	Pre	Rec	F1
CIC IDS 2017	[31]BPE	Sequence of words after BPE encoding	KNN,SVM,RF	0.9954	0.9954	0.9954	0.9954
	[32]DID	Packet arrival time, Raw packet data	LSTM	0.9920	0.9920	0.9980	0.9940
	[33]PBCNN	Raw packet data	CNN	0.9820	0.9820	0.9830	0.9830
	[34]GraphDDoS	Endpoint Traffic Map	GNN	0.9959	0.9965	0.9953	0.9959
	[35]LUCID	Packet space representation	CNN	0.9557	0.9819	0.9318	0.9562
	ERFS	Semantic association graph	MPM	0.9999	0.9999	0.9999	0.9999
CIC DOS 2017	[34]GraphDDoS	Endpoint Traffic Map	GNN	0.9751	0.9505	0.9407	0.9456
	[35]LUCID	Packet space representation	CNN	0.9112	0.9354	0.8845	0.9092
	[36]GNN-NIDS	Host Connection Diagram	GNN	0.9314	0.9110	0.8019	0.8530
	[37]Harun	Seven statistical features such as Fwd Seg Size Min, and Bwd Packets/s	FFCNN	0.9800	0.9750	0.9630	0.9690
	[38]SequenceShield	Directional information of network packets	GRU	0.9692	0.9697	0.9692	0.9694
	ERFS	Semantic association graph	MPM	0.9881	0.9882	0.9880	0.9880
CIC DDoS 2019	[38]SequenceShield	Directional information of network packets	GRU	0.9988	0.9986	0.9986	0.9986
	[39]RTIDS	Sequential feature information	Transformer	0.9858	0.9882	0.9866	0.9848
	[40]DDoSNet	Coding features	RNN	0.9920	0.9928	0.9925	0.9925
	[41]FlowGuard	Ten statistical characteristics such as Flow Byts/s, and Flow Pkts/s.	LSTM	0.9890	0.9947	0.9931	0.9935
	[42]Bi-GAN	Coding features	GAN	0.9968	0.9985	0.9982	0.9984
	ERFS	Semantic association graph	MPM	1.0000	1.0000	1.0000	1.0000
USTC TFC 2016	[2]GCN-Auto	Flow charts containing similarity information	GCN	0.9433	0.9381	0.9354	0.9367
	[43]METC	Coding features	CNN	0.8320	0.7734	0.8541	0.8117
	[1]GCN-TC	Statistical characteristics, flow charts	GCN	0.8924	0.8786	0.8721	0.8753
	[44]DeepFullRange	payload	SAE,LSTM,CNN	0.8748	0.7782	0.8427	0.8236
	[45]DFAE	Raw flow data	CNN,AE	0.9575	0.9675	0.9552	0.9613
	ERFS	Semantic association graph	MPM	0.9787	0.9788	0.9786	0.9787
ISCX VPN- nonVPN	[46]WEAK-SUP	Gray scale flow image	GAN	0.9470	0.8600	0.8100	0.8300
	[47]RAW-SPA	Raw network flow slice data	Resnet,GRU	0.9936	0.9937	0.9936	0.9936
	[48]ICLSTM	Gray scale flow image	LSTM	0.9870	0.9880	0.9900	0.9880
	[49]CNN-ANTLION	CNN middle layer characterization	CNN	0.9800	0.9700	0.9700	0.9700
	[50]CNN-GRU	raw packet data	CNN,RNN	0.9310	0.9350	0.9370	0.9360
	ERFS	Semantic association graph	MPM	0.9964	0.9961	0.9967	0.9964

selection, and feature learning, and most of these models have some similarity with the present method in one or two aspects. Below, a comparative analysis will be conducted from four aspects: detection mechanism, data, feature selection, and feature extraction.

In terms of detection mechanism, analysis of existing advanced models reveals that these models aim to transform the original flow data into a new space in order to investigate the disparities in feature dimensions between normal and abnormal flows. Similar to these models, this method also seeks to map the data into a new feature space. In contrast to existing methods that extract temporal and associative features for analysis, this approach extracts semantic features containing temporal relationships and transmission content by constructing a semantic association graph. Subsequently, it updates the graph information through message passing networks to extract feature representations with spatial characteristics. This detection mechanism enables this method to concurrently extract data feature representations in both time and space dimensions, thereby providing a more comprehensive pattern representation.

On the data side, existing state-of-the-art research has also focused on mining feature information from raw network flow data to characterize the flow using data slicing. Xu and Zhou [30] constructed vocabularies for feature extraction and payload embedding on a single-byte basis based on raw flow data. DID [31] uses the flow's basic content bytes as input to effectively differentiate content-based attacks. PBCNN [32] automatically extracts abstract features from packet bytes in the raw Pcap file, with a subsequent stage constructing representations from packets in the flow or session to fully leverage fundamental data information. METC [42] uses the raw packet data to input the CNN model. Zhao et al. [46] divides the basic network flow into sessions and the packets within each session into 784-byte slices, using the sliced data to describe the flow. Compared with these methods, this approach utilizes the original flow data. It expresses the data at a uniformly fine-grained level based on the semantic idea, retaining as much complete information as possible while describing the behavior at a deeper level.

Regarding feature selection, some existing advanced research starts from the relationship between individual

network flows and behaviors, extracts relationship feature information, and expresses it in graph form. For instance, GraphDDoS [33] transforms flow data into an endpoint traffic graph that encapsulates details about packet relationships (within a single flow) as well as burst and periodic information across multiple flows. Similarly, Pujol-Perich et al. [35] have captured individual flow features and inter-flow relationship features, presenting them through graphical representations. Sun et al. [2] constructed a K-nearest neighbor (KNN) traffic graph containing flow similarity information to represent the flow data structure. They used an autoencoder to form the self representation of the flow data. GCN-TC [1] combines traffic statistical features and traffic tracking maps in the GCN model. ICLSTM [47] converts flow data into ordinary grayscale images and extracts key features using the constructed ICLSTM neural network. Compared with the above research, this method focuses more on the feature information of the network flow itself. By mining the internal structural relationships of the flow data, a semantic association graph is constructed with byte data as nodes and its contextual relationships as edges.

For feature extraction, graph neural networks are commonly used to process graph-structured data by their strong information aggregation ability. Liu et al. [45] converted the flow data into a single-channel grayscale image, which was then fed into a GAN to achieve the learning of the original flow features. Izadi et al. [48] combined CNN, Ant-Lion Heuristic Algorithm (ALO), and Self-Organizing Maps (SOM) for encrypted flow identification. Lin et al. [49] proposed a network structure based on CNN and RNN to learn spatiotemporal features of encrypted flow data. Compared with the above studies, this method focuses more on aggregating neighborhood information. It can use a reduction function to effectively recombine the information in the nodes for update and aggregate it into a semantic representation at the flow level using a self-attention mechanism. The neighborhood information aggregation can help the model capture the contextual information of the nodes better, while the self-attention mechanism allows for precise aggregation, enabling more accurate prediction results.

5) *Generalization Ability Analysis*: In the face of the intricate and constantly evolving network environment, diverse traffic from various backgrounds, varied network topologies, extensive business scenarios, and evolving attack patterns, it is imperative to generalize models. To verify and enhance the generalization of network security methods, this section selects different datasets for cross-testing validation. These datasets exhibit variations in terms of network environment, topology, and business scenarios. By applying the trained model to datasets distinct from the training set, we can assess its performance in new environments and conditions. This process aids in testing the model's capability to identify unknown or unseen network traffic and attack patterns. During the evaluation process, the parameters of the model remain consistent across all evaluation experiments, as detailed in the section on parameter settings.

The effectiveness of this method in detecting various datasets used as training and testing data sources is illustrated

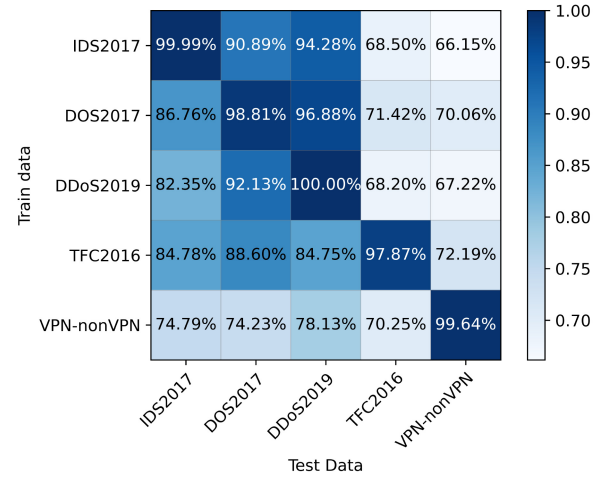


Fig. 6. Classification accuracy of training models on different datasets.

in Figure 6. Generally, the detection performance improves when the business scenarios of the training and testing datasets are more similar. For instance, CIC DOS2017 dataset primarily consists of dos attacks while CIC DDoS2019 dataset mainly contains DDoS attacks, both causing depletion of key resources through a large volume of requests or data to the target system. These datasets share similarities in attack principles, methods, and impact. Consequently, they contain similar attack features and based on traffic characteristics in the training dataset, the model can recognize behaviors in the testing dataset as well.

In the event of substantial disparities between the business scenarios in the training and testing datasets, our method still achieves a detection accuracy of over 65%. For instance, while the CIC IDS2017 dataset encompasses common Internet attacks such as Brute Force FTP, Brute Force SSH, and DoS, the ISCX VPN-nonVPN dataset comprises 7 types of regular encrypted traffic and 7 types of protocol encapsulated flow. The visibility and features of normal network flow differ significantly from those of encrypted flow. The model trained on the ids2017 dataset encounters challenges in applying its learned flow features to the ISCX VPN-nonVPN dataset due to these differences. Moreover, commonly used payload features are ineffective in characterizing encrypted flows because they cannot access packet payload content. In this scenario, packet header features and overall distribution characteristics play a crucial role in identifying encrypted flows, such as packet length and timing. While semantic correlation graphs constructed using our method are effective only for part of the packet header data when it comes to encrypted flows; their effectiveness is limited which results in poorer detection performance.

6) *Learning Efficiency Analysis*: The primary innovation of this approach in terms of model efficiency lies in the proposal of a global weight sharing mechanism for edges and a small window learning mechanism. This eliminates the need to calculate edge weights during the testing phase, allowing for graph learning and updating on a smaller scale. Consequently, this section conducts an analysis of the overall complexity

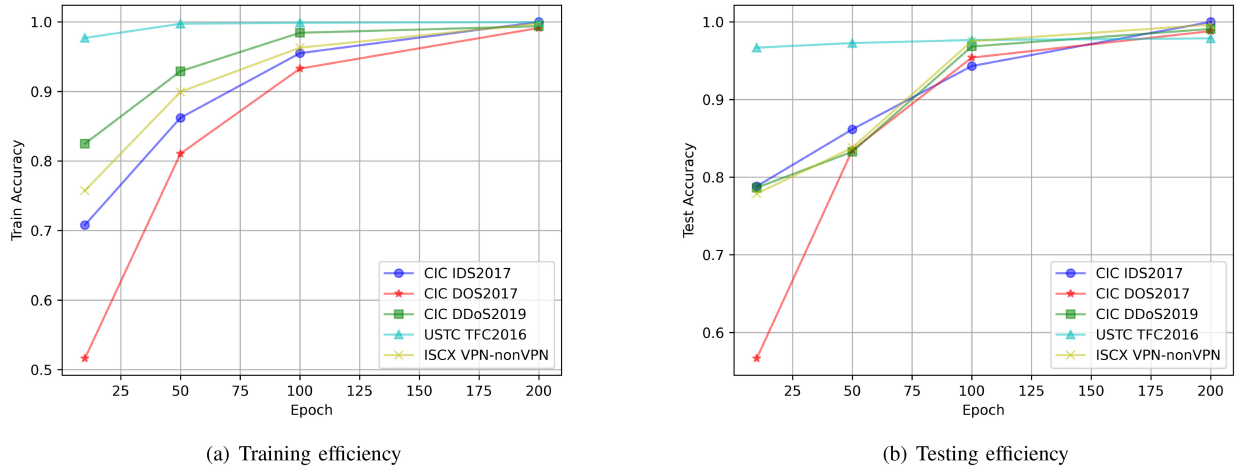


Fig. 7. Analysis of learning efficiency of this method on various datasets.

TABLE VIII
COMPARISON OF LEARNING EFFICIENCY BETWEEN THIS METHOD AND OTHER GRAPH LEARNING METHODS ON VARIOUS DATASETS

Datasets	Composition method	Number of nodes	Number of edges	Graph learning method	Total time (s)	Test accuracy acc under different epochs			
						10	50	100	200
CIC IDS2017	Semantic Association Map	3234334	65536 (511)	GCN	45693	0.4667	0.6571	0.8616	0.9765
				Transformer	103239	0.3963	0.5532	0.8118	0.8905
				MLP	160037	0.5971	0.7657	0.8229	0.8429
				GGNN	78377	0.4533	0.6820	0.7628	0.8135
				ERFS	18386	0.7882	0.8616	0.9431	0.9999
CIC DOS2017	Semantic Association Map	940800	65536 (511)	GCN	13265	0.5348	0.7964	0.9175	0.9643
				Transformer	31840	0.4870	0.6521	0.7659	0.8467
				MLP	44618	0.6098	0.7377	0.8568	0.8890
				GGNN	14380	0.4230	0.6505	0.7236	0.8022
				ERFS	5492	0.5665	0.8347	0.9539	0.9881
CIC DDoS2019	Semantic Association Map	783377	66049 (1021)	GCN	11535	0.5714	0.7971	0.8657	0.9371
				Transformer	26788	0.5357	0.7032	0.7839	0.8876
				MLP	37873	0.6347	0.7429	0.8092	0.8673
				GGNN	13955	0.3347	0.6429	0.7592	0.7773
				ERFS	4694	0.7864	0.8327	0.9683	0.9909
USTC TFC2016	Semantic Association Map	1293600	65536 (511)	GCN	16948	0.5536	0.8007	0.8757	0.9643
				Transformer	52511	0.3218	0.5873	0.6501	0.8764
				MLP	67987	0.5892	0.6664	0.7607	0.8392
				GGNN	35573	0.1392	0.3164	0.4107	0.4392
				ERFS	8784	0.9668	0.9727	0.9766	0.9787
ISCX VPN-nonVPN	Semantic Association Map	1158351	65536 (511)	GCN	11869	0.5192	0.7320	0.8239	0.9045
				Transformer	37674	0.4763	0.6189	0.7455	0.8674
				MLP	47606	0.5095	0.6316	0.7082	0.8548
				GGNN	15478	0.3280	0.4792	0.6288	0.7023
				ERFS	6150	0.7787	0.8379	0.9751	0.9964

and learning efficiency of the method and compares it with graph learning methods such as GCN and Transformer in terms of efficacy. During the evaluation process, consistent data was used, and after constructing the same semantic correlation graph, different graph learning methods such as GCN and Transformer were applied to learn from the semantic correlation graph. Following parameter tuning, the baseline models were configured with a learning rate set to 0.01, batch size at 200, and network depths for GCN, Transformer, MLP, and GGNN models established at 2, 12, 3, and 7 respectively.

In terms of model complexity, we initially conducted a theoretical analysis of the time and space complexity of this approach. The space complexity primarily hinges on the size of the constructed semantic correlation graph, which is directly linked to the number of nodes and edges. Following definitions from previous methods for semantic correlation graphs, its space complexity can be denoted as $O(V_{train} * p + V_{test} * p)$. Here, V_{train} represents the training node set, V_{test} denotes the testing node set, and p signifies the neighborhood connection quantity. With regards to time complexity, we analyze it

across three dimensions: constructing semantic graphs, graph learning, and aggregating node representations. The time complexity for constructing a semantic correlation graph depends on the number of nodes and has a complexity of $O(V_{train} + V_{test})$. During graph learning based on message passing networks, its time complexity is related to the number of node updates and can be represented as $O((V_{train} + V_{test}) * 2)$. After updating learning for testing nodes, it is necessary to use self-attention mechanism to aggregate updated node features into flow-level feature representation; this part has a time complexity of $O((V_{test})^2)$.

The training and testing accuracy of this method under different iterations is shown in Fig. 7. It can be observed that this method only requires a small amount of iterative training to achieve high accuracy, and each iteration requires less time for training, thereby enhancing the overall efficiency of method deployment.

The efficiency comparison analysis between this method and other graph learning methods is shown in Table VIII. Compared with this method, the compared method adopts a unified composition method, but adopts different graph learning methods. To better display performance differences, other graph learning models do not incorporate global edge weight sharing mechanisms and small window learning mechanisms. The hyperparameter settings for all methods remain consistent. Analysis shows that this method has significant advantages in learning efficiency. The byte nodes in this method are only connected to adjacent byte nodes in the network flow data, while other graph learning models connect nodes within a relatively large window. Furthermore, the global sharing of edge weights obviates the necessity for recalculating weight relationships between nodes during the testing process, thereby reducing computational overhead. We also found that this method performs better than other graph learning models. Based on the semantic association graph composition method, the GCN model connects graph nodes within a large window, which cannot distinguish the importance between different byte nodes. In contrast, our proposed method incorporates a self-attention mechanism to enable differential expression of byte data in various network flows. Additionally, the edge weight global sharing mechanism facilitates training of each byte data node across the entire corpus containing all network flow data with consistent configuration.

7) *Application Scenario Analysis*: The evaluation of this model utilized five public datasets, which can be classified into three types of real network detection scenarios: Internet, IoT, and encrypted Internet. Specifically, the CIC IDS2017 dataset represents Internet scenarios, while the CIC DOS2017 and CIC DDoS2019 datasets represent IoT scenarios. The USTC-TFC2016 and ISCX VPN-nonVPN datasets are indicative of encrypted Internet scenarios. The model demonstrated strong detection performance across five datasets. However, notable variations in detection performance were observed among the different datasets. Given that these datasets represent diverse application scenarios, a comprehensive analysis of the model's specific detection performance in various application scenarios is essential. Therefore, this section conducts an in-depth examination of the model's specific detection performance on

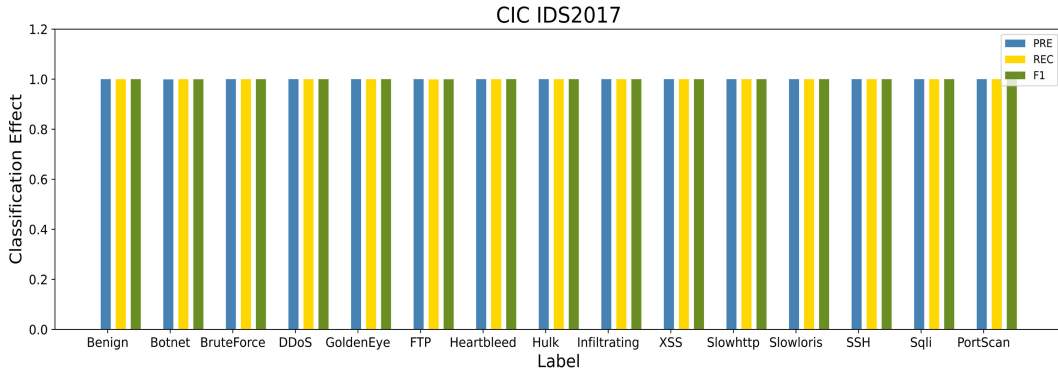
each dataset representing distinct application scenarios and investigates its ability to detect various threats across different datasets, as illustrated in Figure 8.

In the context of the Internet, the CIC IDS2017 dataset encompasses malicious traffic such as Botnet, Brute Force, DDoS, FTP, GoldenEye, Heartbleed, Hulk, infiltration, PortScan, and others. As depicted in Fig. 8, this method demonstrates effective identification of most attacks including Brute Force, HULK, infiltration and Slowloris. These attacks all have prominent behavioral characteristics in the time dimension. Their primary behavioral disparities stem from their varied methods of attack: Brute Force involves attempting weak passwords to gain unauthorized access; HULK generates numerous falsified network flows on the web server to directly target its resource pool; infiltration employs a comprehensive array of tactics for network penetration; and Slowloris utilizes slow HTTP requests to maliciously occupy server connections without releasing them. These attacks present significant variations in flow data content and are better characterized by semantic features based on network flow content.

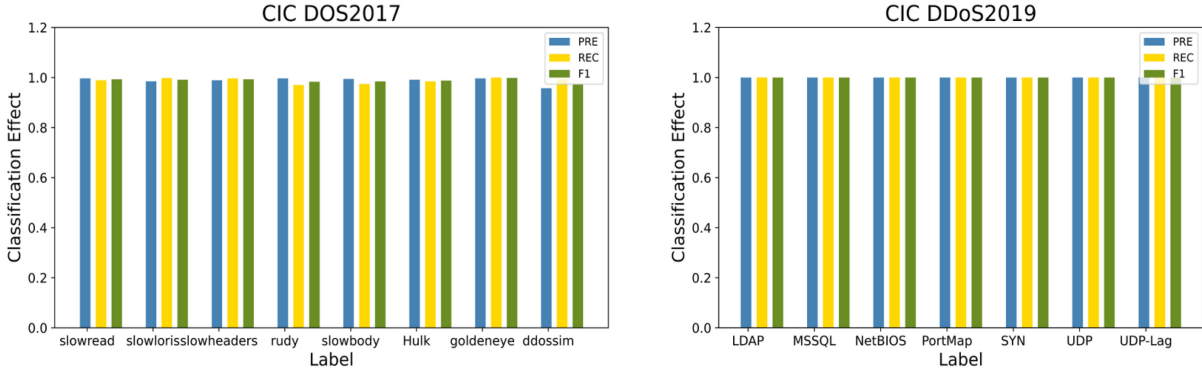
In the IoT scenario, the CIC DOS2017 dataset encompasses eight types of DoS attacks and benign traffic, while the CIC DDoS2019 dataset comprises seven common types of DDoS attacks and mild traffic. Malicious behavior consumes network bandwidth and resources by sending numerous packets and forging the source address of packets, ultimately paralyzing hosts and network systems. In this scenario, denial of service attacks such as DoS and DDoS have obvious spatiotemporal characteristics, with behaviors that possess unique semantic representations. By extracting co-occurrence information to characterize the semantics of traffic behavior, benign network flows can be effectively distinguished from abnormal ones.

For the encrypted Internet scenario, the USTC TFC2016 dataset contains ten types of malicious flows across eight kinds of applications, and the ISCX VPN-nonVPN dataset includes seven types of regular encrypted flows and seven types of protocol encapsulated flows. Unlike regular network flows, encrypted network flows conceal their payload contents. Most existing methods for identifying encrypted flows rely on statistical features such as packet length and packet time, which have been proven to be effective [50], [51]. For some simple encrypted malicious traffic, this method has a good detection effect, such as Zeus, Virut, Tinba, etc. However, its effectiveness is somewhat reduced for Neris and Cridex attacks. This may be due to the fact that the core features of these attacks are not fully captured in flow data. Specifically, the Neris attack operates as a spam zombie network primarily using port 25, making characteristics such as port number, transmission time, and packet quantity crucial. The Cridex attack spreads through removable drives and implants itself in infected systems by modifying registry key values.

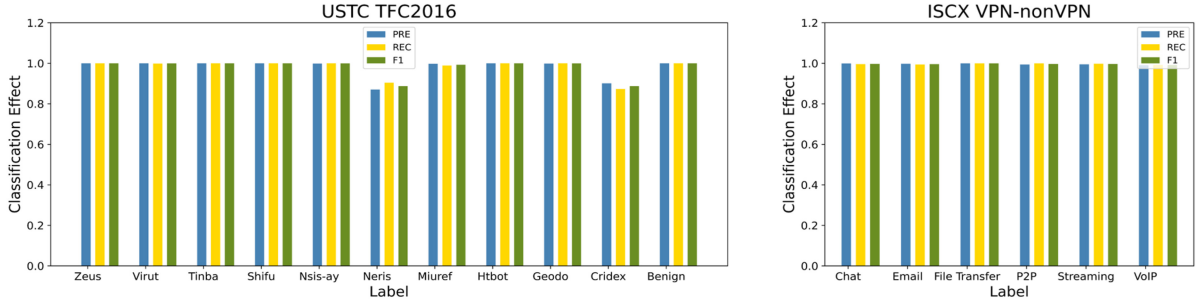
To further investigate the representativeness of raw flow data on behavior, we conducted attention visualization experiments on flow data using the CIC DDoS2019 dataset. Taking a denial-of-service attack as an example, our method achieved a detection accuracy of 93.41% on the CIC DDoS2019 dataset after 50 training iterations, as shown in Fig. 9. Due to limitations in data length, only the initial 30 bytes of data were



(a) Internet. The horizontal axis is as follows: Benign, Botnet, BruteForce, DDoS, GoldenEye, FTP, HeartBleed, Hulk, Infiltrating, XSS, SlowHttp, SlowLoris, SSH, SqlInjection, PortScan.



(b) IoT. The left horizontal axis is as follows: slowread, slowloris, slowheaders, rudy, slowbody, Hulk, goldeneye, ddoSIM. The right horizontal axis is as follows: LDAP, MSSQL, NetBIOS, PortMap, SYN, UDP, UDP-Lag.



(c) Encrypted Internet. The left horizontal axis is as follows: Zeus, Virut, Tinba, Shifu, Nsis-ay, Neris, Miuref, Htbot, Geodo, Cridex, Benign. The right horizontal axis is as follows: Chat, Email, FileTransfer, P2P, Streaming, VoIP.

Fig. 8. The classification performance of this method in different application scenarios.

extracted. As the raw network flow data passes through the attention layer of the message passing network, it demonstrates varying role weights. Notably, in IoT scenarios, the high-weighted attention values primarily focus on payload data, with increased emphasis on recurring data content. Combined with the principle and process of a DDoS attack, this method can more effectively uncover behavioral patterns by capturing semantic content relationships within the raw flow data.

V. CONCLUSION

This study presents a novel threat detection method based on semantic association graphs. In this method, flow data is

treated as a specific language and semantic correlation graphs are constructed to capture the inherent semantic correlations within the flow data. To enhance the effectiveness of our method, we have designed a global sharing mechanism that enables the sharing of node representations and edge weights. Utilizing a messaging network facilitates small-scale graph learning and updating, which are then aggregated into flow-level semantic representations for threat detection and behavior recognition. The final evaluation results demonstrate the effectiveness of our method in extracting flow data features across various scenarios, achieving high classification accuracies of 99.99%, 98.81%, 100%, 97.87%, and 99.64% on the CIC IDS2017, CIC DOS2017, CIC DDoS 2019, USTC TFC2016, and ISCX VPN-non VPN datasets.

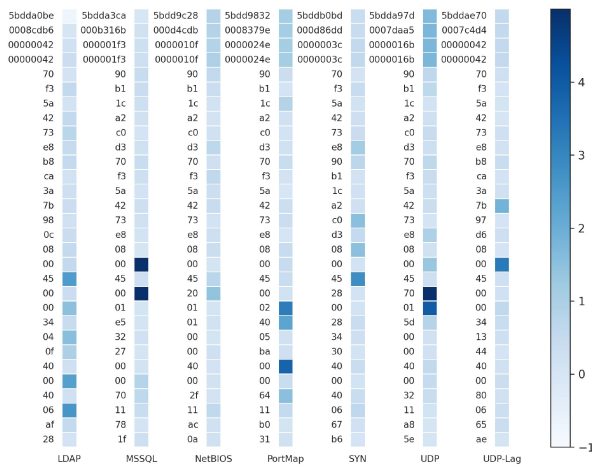


Fig. 9. Attention Visualization of flow Data in CIC DDoS2019 Dataset.

This method is currently only suitable for more straightforward encrypted Internet scenarios, but it has some ability to detect encrypted network threats. In future research, we plan to create a model specifically for encrypted network traffic scenarios to meet different threat detection needs. We will also focus on tracking and tracing threats to strengthen defense capabilities and improve security strategies.

REFERENCES

- [1] J. Zheng and D. Li, "GCN-TC: Combining trace graph with statistical features for network traffic classification," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2019, pp. 1–6.
- [2] B. Sun, W. Yang, M. Yan, D. Wu, Y. Zhu, and Z. Bai, "An encrypted traffic classification method combining graph convolutional network and autoencoder," in *Proc. IEEE 39th Int. Perform. Comput. Commun. Conf. (IPCCC)*, 2020, pp. 1–8.
- [3] L. Zhang, H. Shi, K. Zhang, H. Sun, and W. Zhang, "GraphMAL: A network malicious traffic identification method based on graph neural network," in *Proc. Int. Conf. Netw. Netw. Appl. (NaNA)*, 2023, pp. 262–267.
- [4] Y. Zhang, Y. Li, W. Deng, K. Huang, and C. Yang, "Complex networks identification using Bayesian model with independent laplace prior," *Chaos*, vol. 31, no. 1, 2021, Art. no. 13107.
- [5] J. Zheng, Z. Zeng, T. Feng, and L. Maglaras, "GCN-ETA: High-efficiency encrypted malicious traffic detection," *Security Commun. Netw.*, vol. 2022, no. 1, pp. 1–11, Jan. 2022.
- [6] J. Zhao, X. Liu, Q. Yan, B. Li, M. Shao, and H. Peng, "Multi-attributed heterogeneous graph convolutional network for BOT detection," *Inf. Sci.*, vol. 537, pp. 380–393, Oct. 2020.
- [7] N. Okui, Y. Akimoto, A. Kubota, and T. Yoshida, "A graph construction method for anomalous traffic detection with graph neural networks using sets of flow data," in *Proc. IEEE 47th Annu. Comput. Softw. Appl. Conf. (COMPSAC)*, 2023, pp. 1017–1018.
- [8] X. Zhong and G. Wan, "Six-graphsecurity: Industrial Internet intrusion detection based on graph neural network," in *Proc. IEEE 7th Inf. Technol. Mechatron. Eng. Conf. (ITOEC)*, vol. 7, 2023, pp. 1340–1344.
- [9] L. Zhang, L. Tan, H. Shi, H. Sun, and W. Zhang, "Malicious traffic classification for IoT based on graph attention network and long short-term memory network," in *Proc. 24th Asia-Pac. Netw. Oper. Manag. Symp. (APNOMS)*, 2023, pp. 54–59.
- [10] R. Ge, Y. Zhang, C. Si, G. Zhou, and W. Zhou, "MateGraph: Toward mobile malware detection through traffic behavior graph," in *Proc. IEEE 24th Int. Conf. High Perform. Comput. Commun. 8th Int. Conf. Data Sci. Syst. 20th Int. Conf. Smart City 8th Int. Conf. Depend. Sensor Cloud Big Data Syst. Appl. (HPCC/DSS/SmartCity/DependSys)*, 2022, pp. 801–809.
- [11] X. Deng, J. Zhu, X. Pei, L. Zhang, Z. Ling, and K. Xue, "Flow topology-based graph convolutional network for intrusion detection in label-limited IoT networks," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 1, pp. 684–696, Mar. 2023.
- [12] Y. Wang, Y. Shang, Y. He, Y. Li, and J. Liu, "BotMark: Automated BotNet detection with hybrid analysis of flow-based and graph-based traffic behaviors," *Inf. Sci.*, vol. 511, pp. 284–296, Feb. 2020.
- [13] Z. Chen, D. Chen, X. Zhang, Z. Yuan, and X. Cheng, "Learning graph structures with transformer for multivariate time-series anomaly detection in IoT," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9179–9189, Jun. 2022.
- [14] Z. Chen, G. Cheng, D. Niu, X. Qiu, Y. Zhao, and Y. Zhou, "WFF-EGNN: Encrypted traffic classification based on weaved flow fragment via ensemble graph neural networks," *IEEE Trans. Mach. Learn. Commun. Netw.*, vol. 1, pp. 389–411, 2023.
- [15] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.
- [16] X. Yun, Y. Wang, Y. Zhang, and Y. Zhou, "A semantics-aware approach to the automated network protocol identification," *IEEE/ACM Trans. Netw.*, vol. 24, no. 1, pp. 583–595, Feb. 2016.
- [17] Y. Liu, N. Xi, and Y. Zhi, "NLEU: A semantic-based taint analysis for vetting apps in Android," in *Proc. Int. Conf. Netw. Netw. Appl. (NaNA)*, 2021, pp. 327–333.
- [18] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 1263–1272.
- [19] T. Xu and P. Zhou, "Feature extraction for payload classification: A byte pair encoding algorithm," in *Proc. IEEE 8th Int. Conf. Comput. Commun. (ICCC)*, 2022, pp. 1–5.
- [20] M. Soltani, M. J. Siavoshani, and A. H. Jahangir, "A content-based deep intrusion detection system," *Int. J. Inf. Security*, vol. 21, no. 3, pp. 547–562, Jun. 2022.
- [21] L. Yu et al., "PBCNN: Packet bytes-based convolutional neural network for network intrusion detection," *Comput. Netw.*, vol. 194, Jul. 2021, Art. no. 108117.
- [22] Y. Li et al., "GraphDDoS: Effective DDoS attack detection using graph neural networks," in *Proc. IEEE 25th Int. Conf. Comput. Supported Cooper. Work Design (CSCWD)*, 2022, pp. 1275–1280.

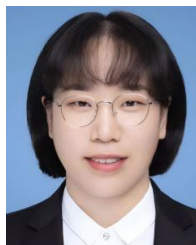
- [34] R. Doriguzzi-Corin, S. Millar, S. Scott-Hayward, J. Martínez-del Rincón, and D. Siracusa, "LUCID: A practical, lightweight deep learning solution for DDoS attack detection," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 2, pp. 876–889, Jun. 2020.
- [35] D. Pujol-Perich, J. Su'arez-Varela, A. Cabellos-Aparicio, and P. Barlet-Ros, "Unveiling the potential of graph neural networks for robust intrusion detection," *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 49, no. 4, pp. 111–117, 2021.
- [36] H. S. Ilango, M. Ma, and R. Su, "A feedforward-convolutional neural network to detect low-rate dos in IoT," *Eng. Appl. Artif. Intell.*, vol. 114, Sep. 2022, Art. no. 105059.
- [37] Z. Deng et al., "SequenceShield: A robust and accurate DDoS detection method via serializing the traffic with direction information," in *Proc. IEEE 24th Int. Conf. High Perform. Comput. Commun. 8th Int. Conf. Data Sci. Syst. 20th Int. Conf. Smart City 8th Int. Conf. Depend. Sensor Cloud Big Data Syst. Appl. (HPCC/DSS/SmartCity/DependSys)*, 2022, pp. 499–507.
- [38] Z. Wu, H. Zhang, P. Wang, and Z. Sun, "RTIDS: A robust transformer-based approach for intrusion detection system," *IEEE Access*, vol. 10, pp. 64375–64387, 2022.
- [39] M. S. Elsayed, N.-A. Le-Khac, S. Dev, and A. D. Jurcut, "DDoSNet: A deep-learning model for detecting network attacks," in *Proc. IEEE 21st Int. Symp. World Wireless Mobile Multimedia Netw. (WoWMoM)*, 2020, pp. 391–396.
- [40] Y. Jia, F. Zhong, A. Alrawais, B. Gong, and X. Cheng, "FlowGuard: An intelligent edge defense mechanism against IoT DDoS attacks," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9552–9562, Oct. 2020.
- [41] W. Xu, J. Jang-Jaccard, T. Liu, F. Sabrina, and J. Kwak, "Improved bidirectional GAN-based approach for network intrusion detection using one-class classifier," *Computers*, vol. 11, no. 6, p. 85, 2022.
- [42] G. Aceto, D. Ciunzo, A. Montieri, and A. Pescapé, "Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges," *IEEE Trans. Netw. Service Manag.*, vol. 16, no. 2, pp. 445–458, Jun. 2019.
- [43] Y. Zeng, H. Gu, W. Wei, and Y. Guo, "deep – full – range: A deep learning based network encrypted traffic classification and intrusion detection framework," *IEEE Access*, vol. 7, pp. 45182–45190, 2019.
- [44] M. He, X. Wang, J. Zhou, Y. Xi, L. Jin, and X. Wang, "Deep-feature-based autoencoder network for few-shot malicious traffic detection," *Security Commun. Netw.*, vol. 2021, no. 6, pp. 1–13, Mar. 2021.
- [45] Liu et al., "A weakly-supervised method for encrypted malicious traffic detection," in *Proc. Int. Symp. Grids Clouds PoS(ISGC)*, 2022, pp. 1–9.
- [46] G. Zhao, Z. X. Wang, and Z. Yang, "An encrypted traffic classification model based on the raw traffic and spatiotemporal characteristics," in *Proc. 6th Int. Conf. Electron. Inf. Technol. Comput. Eng.*, 2022, pp. 1208–1213.
- [47] B. Lu, N. Luktarhan, C. Ding, and W. Zhang, "ICLSTM: Encrypted traffic service identification based on inception-LSTM neural network," *Symmetry*, vol. 13, no. 6, p. 1080, 2021.
- [48] Izadi, M. Ahmadi, and R. Nikbazz, "Network traffic classification using convolutional neural network and ant-lion optimization," *Comput. Elect. Eng.*, vol. 101, Jul. 2022, Art. no. 108024.
- [49] C. Y. Lin, B. Chen, and W. Lan, "An efficient approach for encrypted traffic classification using CNN and bidirectional GRU," in *Proc. 2nd Int. Conf. Consum. Electron. Comput. Eng. (ICCECE)*, 2022, pp. 368–373.
- [50] T. Shapira and Y. Shavitt, "FlowPIC: A generic representation for encrypted traffic classification and applications identification," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1218–1232, Jul. 2021.
- [51] W. Chen, F. Lyu, F. Wu, P. Yang, G. Xue, and M. Li, "Sequential message characterization for early classification of encrypted Internet traffic," *IEEE Trans. Veh. Technol.*, vol. 70, no. 4, pp. 3746–3760, Apr. 2021.



Yiqing Luo is currently pursuing the Doctoral degree with the School of Electronic Engineering, Beijing University of Posts and Telecommunications, China. Her research interests include behavior recognition, anomaly detection, and deep learning.



Mingshu He (Member, IEEE) received the Ph.D. degree from the Beijing University of Posts and Telecommunications, Beijing, China, in 2022, where he is currently an Assistant Professor with the School of Cyberspace Security. His research interests include network security, anomaly detection, and machine learning.



Xiaojuan Wang received the Ph.D. degree in electronic science and technology from the Beijing University of Posts and Telecommunications, where she is currently an Associate Professor with the School of Cyberspace Security. Her research interests include deep learning, complex networks, and human gesture recognition.